

Институт (факультет)
Кафедра

Институт информационных технологий
Математики и информатики

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ

Выполнил студент группы

1ПМб-04-1оп-23

группа

направления подготовки (специальности)

Прикладная математика и информатика

шифр, наименование

Селянский Владислав Николаевич

фамилия, имя, отчество

Руководитель

Лашенков Денис Игоревич

фамилия, имя, отчество

Ассистент кафедры МиИ

Должность

Дата представления работы

« _____ » _____ 20 ____ г.

Оценка _____, _____
количество баллов

Подпись преподавателя _____

Череповец, 2025
год

Оглавление

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ	1
Введение	3
Раздел первый: «Язык R и среда разработки R Studio»	4
Задание 1. Первый сеанс RStudio.	4
Задание 2. Работа с функциями в языке R.	7
Задание 3. Структуры данных в R.	9
Задание 4. Управление данными в R.	11
Задание 5. Визуализация данных в R.	13
Раздел второй: «Отчет о выполнении индивидуального задания»	19
Разработка предиктивной модели для определения вероятности заболевания диабетом.	19
Описание используемых инструментов.....	20
Описание используемой модели.....	21
Описание набора данных.....	21
Описание используемых метрик	21
Реализация решения.....	22
Подключение необходимых пакетов	22
Чтение и предобработка данных	22
Обучение и предсказание модели	25
Определение порогового значения	25
Определение классов и векторов с метками	27
Использование различных метрик точности модели	27
Сохранение модели.....	28
Заключение	29
Список литературы	30

Введение

Прохождение учебной практики: технологическая (проектно-технологическая) практика имеет цель формирование способности:

- управлять своим временем, выстраивать и реализовывать траекторию саморазвития на основе принципов образования в течение всей жизни;
- разрабатывать и применять методы машинного обучения для решения практических задач;
- использовать инструментальные средства для решения задач машинного обучения;
- осуществлять сбор и подготовку данных для систем искусственного интеллекта.

При этом решаются задачи:

- закрепления и расширения знаний, полученных при изучении базовых дисциплин первого и второго курсов, повышения общей и профессиональной эрудиции;
- сбора и анализа теоретического и справочного материала для выполнения индивидуального задания;
- изучения языка R и среды разработки RStudio;
- осуществления поиска данных, их подготовки для выполнения практического задания;
- выбора методов машинного обучения для выполнения практического задания;
- определения метрики оценки результатов использования методов машинного обучения в рамках выполнения практического задания;
- разработки модели машинного обучения для выполнения практического задания;
- формулирования теоретических и практических выводов на основе критического переосмысления накопленного опыта.

Раздел первый: «Язык R и среда разработки R Studio»

Задание 1. Первый сеанс RStudio.

1. Запустите RStudio Cloud (<https://rstudio.cloud>). Пройдите регистрацию.
2. В своем рабочем пространстве создайте новый проект PracticeTask1.
3. Создайте простой набор данных (в терминологии R — вектор), состоящий из чисел 1, 2 и 4, и присвойте ему имя x. Выведите на экран фразу «Это мой первый сеанс работы с R».

```
x <- c(1, 2, 4)
x
## [1] 1 2 4
print("Это мой первый сеанс работы с R")
## [1] "Это мой первый сеанс работы с R"
```

4. Выполните команду, которая присваивает q значение (1,2,4,1,2,4,8), используя ранее определенный вектор x.

```
q <- c(x, x, 8)
q
## [1] 1 2 4 1 2 4 8
```

5. Выведите на экран значение 3-го элемента вектора x.

```
x[3]
## [1] 4
```

6. Выведите на экран элементы вектора x со 2 по 3

```
x[2:3]
## [1] 2 4
```

7. Выведите на экран математическое ожидание и среднеквадратическое отклонение для элементов вектора x.

```
mean(x)
```

```
## [1] 2.333333
```

```
sd(x)
```

```
## [1] 1.527525
```

8. Выведите на экран перечень внутренних наборов данных, предназначенных для демонстраций.

```
data()
```

9. Вычислите математическое ожидание и среднее квадратическое отклонение для набора данных Nile (этот набор содержит данные о течении Нила).

```
mean(Nile)
```

```
## [1] 919.35
```

```
sd(Nile)
```

```
## [1] 169.2275
```

10. Выведите гистограмму этих данных (См. рисунок 1):

```
hist(Nile)
```

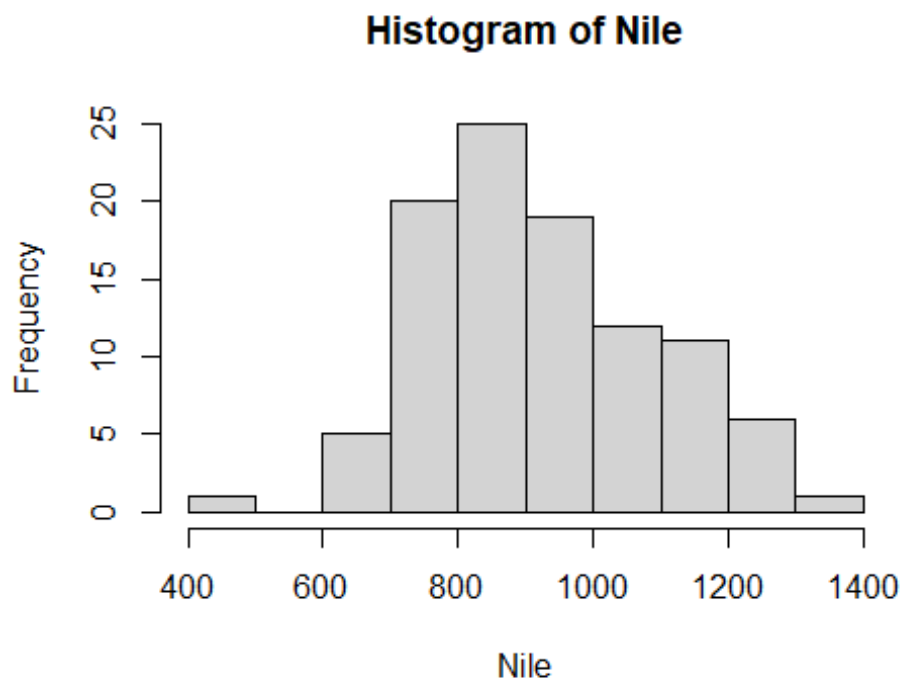


Рисунок 1. Гистограмма течений Нила.

11. Сохраните проект.

12. Ответьте на вопросы:

1) Как сформировать вектор?

Ответ: Вектор формируется с помощью функции `c()`.

2) Что означает операция `c` (`concatenate`)?

Ответ: Операция `c` (`concatenate`) объединяет элементы в один вектор.

3) Как вывести на экран значение переменной величины?

Ответ: Для вывода значения переменной используется функция `print()`.

4) Нужно ли указывать тип переменной величины?

Ответ: Нет, в R тип переменной определяется автоматически.

5) Как вывести на экран перечень демонстрационных наборов данных?

Ответ: Для вывода перечня демонстрационных наборов данных используется функция `data()`.

6) Как рассчитать среднеквадратическое отклонение?

Ответ: Среднеквадратическое отклонение рассчитывается с помощью функции `sd()`.

7) Как рассчитать математическое ожидание?

Ответ: Математическое ожидание рассчитывается с помощью функции `mean()`.

8) Как построить гистограмму?

Ответ: Гистограмма строится с помощью функции `hist()`.

Задание 2. Работа с функциями в языке R.

1. Запустите RStudio Cloud. Создайте новый проект PracticeTask2 в своем рабочем пространстве.
2. Определите функцию с именем oddcount(), предназначенную для подсчета нечетных чисел в целочисленном векторе:

```
# Функция подсчитывает количество нечетных целых чисел в x
oddcount <- function(x) {
  k <- 0 # в начале вычислений количество нечетных чисел равно 0
  for (n in x) {
    if (n %% 2 == 1) {# %% - вычисление остатка
      k <- k+1
    }
  }
  return(k)
}
```

3. Выполните команду Code – Source File...
4. В режиме интерпретатора вычислите количество нечетных чисел в сформированных в предыдущей работе наборах данных x и q с использованием созданной функции:

```
print(oddcount(x))
## [1] 1
print(oddcount(q))
## [1] 2
```

5. Попробуйте вывести в интерпретаторе значения переменных k и n. Что Вы получили?

```
Error:
! объект 'k' не найден
```

6. Определите функцию sum_2, складывающую два числа, одно из которых имеет значение по умолчанию, равное 5 Протестируйте работу функции. Сохраните файл в своей папке под именем lab2_2.

```
sum_2 <- function(a, b = 5)
{
  return(a + b)
}

print(sum_2(3))
## [1] 8

print(sum_2(3, 8))
## [1] 11
```

7. Напишите функцию fx, определенную на всей числовой прямой
Протестируйте работу функции.

```
fx <- function(x) ifelse(x <= -2, 1-(x+2)^2, ifelse(x <= 2, -x/2, (x-2)^2 + 1))

print(fx(4.5))
## [1] 7.25

print(fx(-4.5))
## [1] -5.25

print(fx(1))
## [1] -0.5
```

8. Сохраните проект.

9. Ответьте на вопросы:

1) Как определить функцию?

Ответ: Функция определяется с помощью ключевого слова function, например:
my_function <- function(arg1, arg2) {}

2) Как описать тело функции?

Ответ: Тело функции описывается в фигурных скобках {}, где размещается код, который будет выполняться.

3) Как записывается оператор if в языке R?

Ответ: if записывается как: if (условие) { действия }

4) Как записывается оператор for в языке R?

Ответ: for записывается как: for (i in 1:10) { действия }

5) Какие переменные являются глобальными?

Ответ: Глобальными являются переменные, определенные вне функций и доступные во всем скрипте.

6) Какие переменные являются локальными по отношению к функции?

Ответ: Локальными являются переменные, определенные внутри функции и доступные только в ее теле.

7) Как определить значения по умолчанию для аргументов функции?

Ответ: Значения по умолчанию для аргументов функции определяются с помощью знака = в определении функции, например: function(arg1 = значение)

8) В каком порядке должны располагаться аргументы функции, не имеющие значений по умолчанию, и аргументы, имеющие значения по умолчанию?

Ответ: Аргументы без значений по умолчанию должны располагаться перед аргументами с значениями по умолчанию.

Задание 3. Структуры данных в R.

1. Запустите RStudio Cloud. В своем рабочем пространстве создайте новый проект PracticeTask3.

2. Создайте вектор a1 из вектора a1 получите вектор a1, добавив в середину вектора число 161.

```
a <- c(34, 5, 73, 12)
a1 <- c(a[1:(length(a)/2)], 161, a[(length(a)/2):length(a)])
a1
## [1] 34 5 161 5 73 12
```

3. Создайте фактор month1, содержащий названия весенних месяцев года, при этом должен быть задан полный перечень месяцев и установлен порядок следования месяцев.

```
month1 <- factor(c("Март", "Апрель", "Май"), levels = c("Март", "Апрель", "Май"), ordered = T)
month1

## [1] Март  Апрель  Май
## Levels: Март < Апрель < Май
```

4. Сформируйте список emp, содержащий сведения о сотруднике: имя (name – имеет символьные значения); зарплата (salary – вещественное число), принадлежность к профсоюзу (union – логическое значение). Внесите данные об одном сотруднике: Irina, 40800.00, F. Выведите зарплату сотрудника. Измените зарплату сотрудника, задав значение 50000.00.

```
emp <- list(name = "Irina",
            salary = 40800.00,
            union = F)
emp

## $name
## [1] "Irina"
##
## $salary
## [1] 40800
##
## $union
## [1] FALSE
```

5. Создайте фрейм department, описав сотрудников отдела атрибутами: name; salary; union. Введите данные по 2-м сотрудникам. Выведите имена всех сотрудников. Выведите зарплату второго сотрудника.

```
department <- data.frame(name = "Sergey",
                        salary = 42000,
                        union = T)
department <- rbind(department, emp)

print(department)

##      name salary union
## 1 Sergey  42000  TRUE
## 2  Irina  40800 FALSE
```

```
print(department$name)
## [1] "Sergey" "Irina"
print(department[2, 2])
## [1] 40800
```

6. Создайте матрицу `m` размером 3x2.

```
m <- matrix(1:6, 3, 2)
```

7. Сохраните проект.

Задание 4. Управление данными в R.

1. Откройте файл `PracticeTask3`, созданный как результат выполнения предыдущего задания, сохраните объекты с именами `subject1`, `department`, `month1` в файл `mydata.RData`.

```
save(subject1, department, month1, file = "mydata.RData")
```

2. Создайте новый файл с именем `lab2.R` и загрузите в него объекты из файла `mydata_lab1.RData`, выведите объекты `subject1`, `department`, `month1`.

```
rm(list = ls())
load("mydata.RData")
```

3. Сохраните текущий сеанс, воспользовавшись командой `save.image()`.

```
save.image()
```

4. Используя функцию `rm()` удалите объекты `month1` и `subject1`, проверьте список оставшихся объектов, используя функцию `ls()`. Удалите все объекты. Используя файл `.RData`, восстановите объекты.

```
rm(month1, subject1)
ls()
## [1] "department"
rm(list = ls())
ls()
## character(0)
load("mydata.RData")
```

5. Откройте файл R, созданный в ходе выполнения предыдущего задания. Запишите в файл pt_data.csv фрейм данных pt_data, определив параметр row.names=FALSE. Вернитесь в файл R текущей работы. Прочитайте содержимое файла pt_data.csv.

```
pt_data <- read.csv("pt_data.csv", stringsAsFactors = F)
write.csv(pt_data, file = "pt_data.csv", row.names = F)
```

6. Загрузите файл usedcars.csv на сервер, если Вы используете среду RStudio Cloud, используя кнопку.

7. Загрузите данные из файла usedcars.csv во фрейм usedcars.

```
usedcars <- read.csv("usedcars.csv")
```

8. Отобразите структуру usedcars, используя команду str().

```
str(usedcars)
## 'data.frame':    150 obs. of  6 variables:
## $ year          : int  2011 2011 2011 2011 2012 2010 2011 2010 2011 2010 ...
## $ model         : chr  "SEL" "SEL" "SEL" "SEL" ...
## $ price         : int  21992 20995 19995 17809 17500 17495 17000 16995 16995 16995 ..
## $ mileage       : int  7413 10926 7351 11613 8367 25125 27393 21026 32655 36116 ...
## $ color         : chr  "Yellow" "Gray" "Silver" "Gray" ...
## $ transmission: chr  "AUTO" "AUTO" "AUTO" "AUTO" ...
```

9. Используя функцию summary() запросите статистику по всем числовым переменным usedcars.

```
summary(usedcars)
##      year      model      price      mileage
## Min.   :2000   Length:150   Min.    : 3800   Min.     : 4867
## 1st Qu.:2008   Class :character 1st Qu.:10995   1st Qu.: 27200
## Median :2009   Mode  :character Median :13592   Median : 36385
## Mean   :2009                Mean  :12962   Mean   : 44261
## 3rd Qu.:2010                3rd Qu.:14904   3rd Qu.: 55125
## Max.   :2012                Max.    :21992   Max.    :151479
##      color      transmission
## Length:150      Length:150
## Class :character Class :character
## Mode  :character Mode  :character
##
```

```
##  
##
```

10. Используя функцию `mean()` посчитайте средние значения для всех числовых переменных `usedcars`.

```
sapply(usedcars[, c(3, 4)], mean, rm.na = T)  
## price mileage  
## 12961.93 44260.65
```

11. Используя функцию `median()` посчитайте медианы для всех числовых переменных `usedcars`.

```
sapply(usedcars[, c(3, 4)], median, rm.na = T)  
## price mileage  
## 13591.5 36385.0
```

12. Изучите пятичисловую сводку для переменных `price` и `mileage`.

```
summary(usedcars$price)  
## Min. 1st Qu. Median Mean 3rd Qu. Max.  
## 3800 10995 13592 12962 14904 21992  
  
summary(usedcars$mileage)  
## Min. 1st Qu. Median Mean 3rd Qu. Max.  
## 4867 27200 36385 44261 55125 151479
```

Задание 5. Визуализация данных в R.

1. Рассмотрим диаграмму размаха для данных о цене и пробеге подержанных автомобилей. Чтобы получить диаграмму размаха для переменной, воспользуемся функцией `boxplot()` (См. рисунок 2 и рисунок 3).

```
boxplot(usedcars$price, main = "Boxplot of Used Car Prices", ylab = "Price ($)")
```

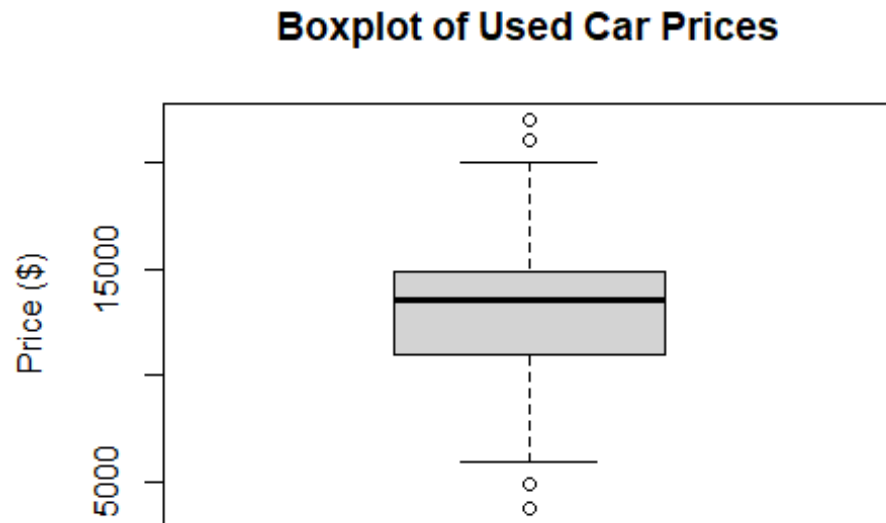


Рисунок 2. Ящик с усами для цен.

```
boxplot(usedcars$mileage, main = "Boxplot of Used Car Mileage",  
        ylab = "Odometer (mi.)")
```

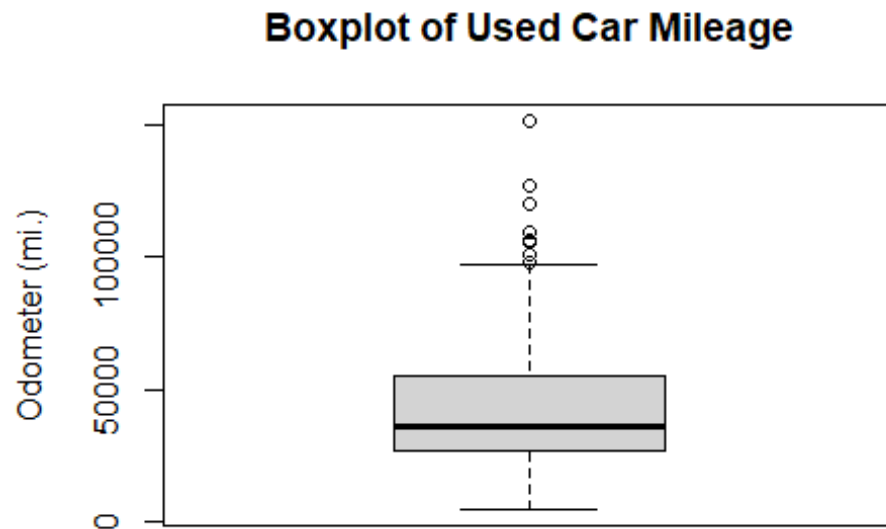


Рисунок 3. Ящик с усами для пробега.

2. Для того чтобы построить гистограмму для данных о цене и пробеге подержанных автомобилей, можно воспользоваться функцией `hist()` (См. рисунок 4 и рисунок 5).

```
hist(usedcars$price, main = "Histogram of Used Car Prices", xlab = "Price ($)")
```

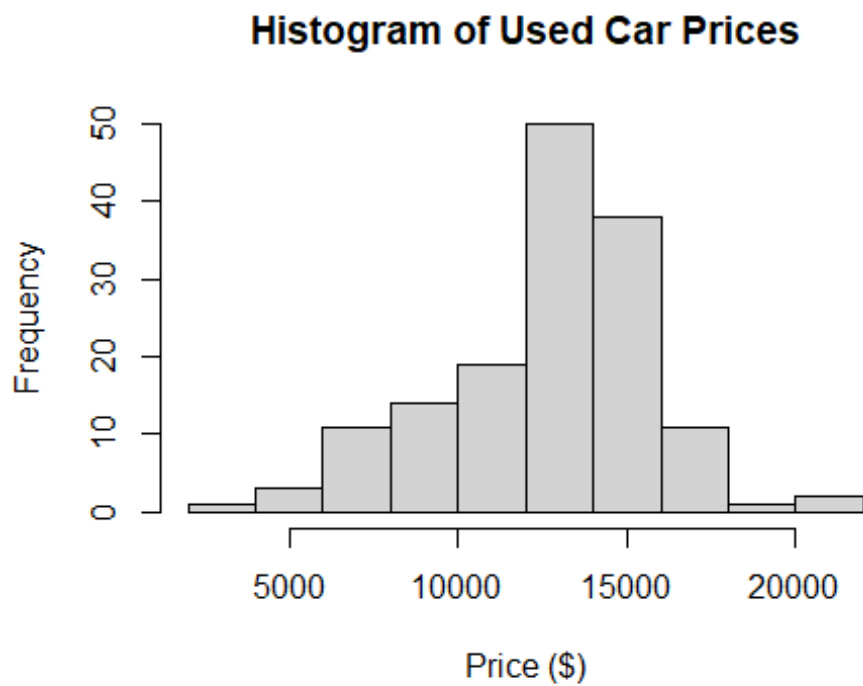


Рисунок 4. Гистограмма цен.

```
hist(usedcars$mileage, main = "Histogram of Used Car Mileage",  
     xlab = "Odometer (mi.)")
```

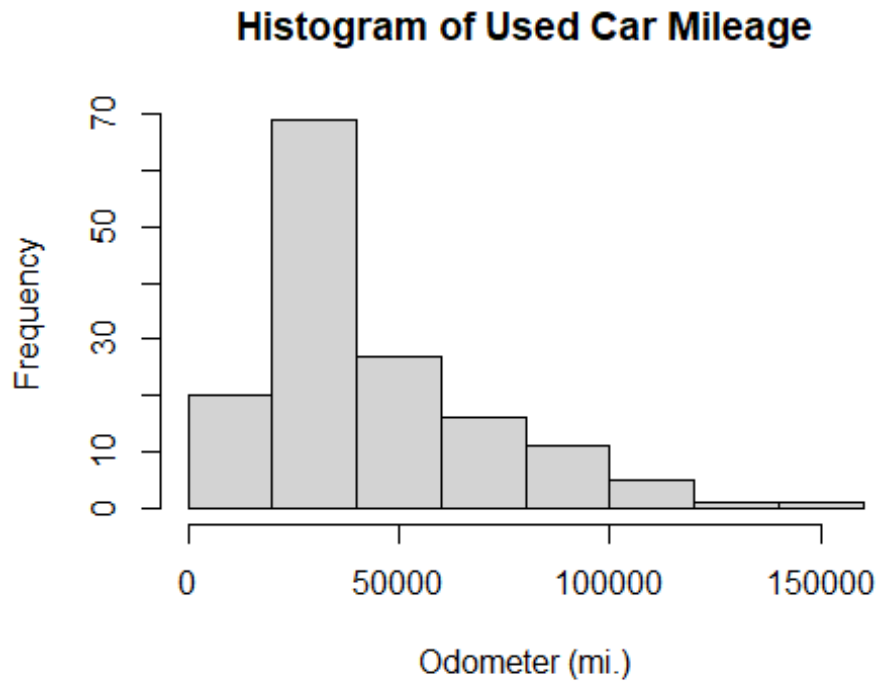


Рисунок 5. Гистограмма пробега.

3. В R для вычисления дисперсии и стандартного отклонения используются функции `var()` и `sd()`.

```
print(var(usedcars$price))  
## [1] 9749892  
print(sd(usedcars$price))  
## [1] 3122.482  
print(var(usedcars$mileage))  
## [1] 728033954  
print(sd(usedcars$mileage))  
## [1] 26982.1
```

4. Чтобы построить таблицу частотности для данных о поддержанном автомобиле, можно воспользоваться функцией `table()`.

```
print(table(usedcars$year))
```



```
##
## 2000 2001 2002 2003 2004 2005 2006 2007 2008 2009 2010 2011 2012
##    3    1    1    1    3    2    6   11   14   42   49   16    1

print(table(usedcars$model))

##
##  SE  SEL  SES
##  78   23   49

print(table(usedcars$color))

##
##  Black   Blue   Gold   Gray   Green   Red Silver  White Yellow
##    35    17     1    16     5     25    32    16     3
```

5. R также позволяет вычислить пропорции таблицы напрямую, с помощью команды `prop.table()` для таблицы, созданной функцией `table()`.

```
model_table <- table(usedcars$model)
prop.table(model_table)

##
##          SE          SEL          SES
## 0.5200000 0.1533333 0.3266667
```

6. Результаты `prop.table()` можно объединять с другими функциями R для представления вывода в другой форме. Предположим, что мы хотим отобразить результаты в процентах с одним десятичным знаком.

```
color_table <- table(usedcars$color)
color_pct <- prop.table(color_table) * 100
round(color_pct, digits = 1)

##
##  Black   Blue   Gold   Gray   Green   Red Silver  White Yellow
##   23.3   11.3    0.7   10.7    3.3   16.7   21.3   10.7    2.0
```

7. Чтобы ответить на вопрос о соотношении цены и пробега, построим диаграмму разброса. Для этого воспользуемся функцией `plot()` с параметрами `main`, `xlab` и `ylab` (См. рисунок 6).

```
plot(x = usedcars$mileage, y = usedcars$price,
     main = "Scatterplot of Price vs. Mileage",
     xlab = "Odometer", ylab = "Price")
```

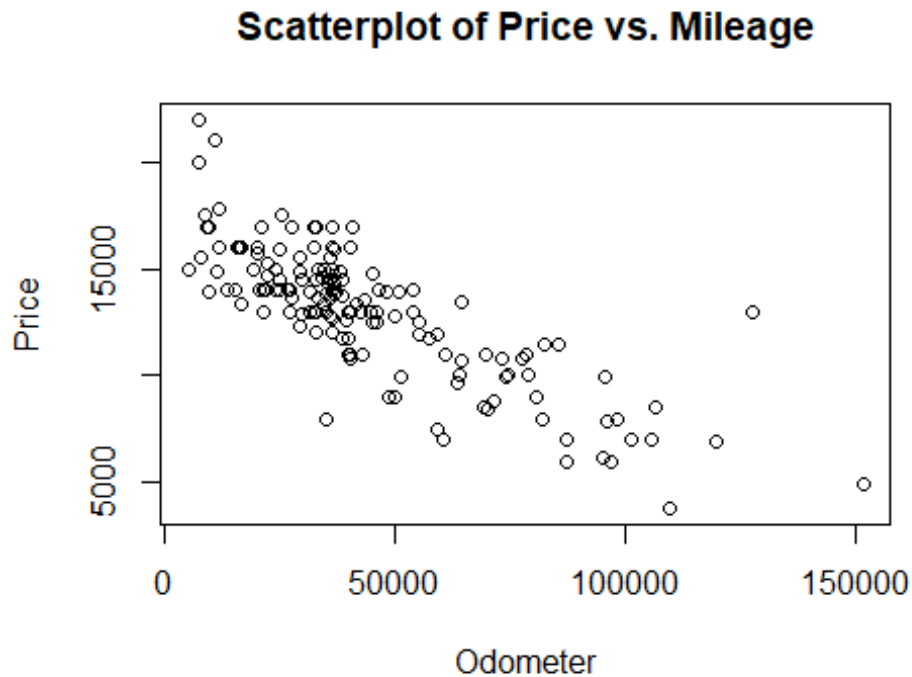


Рисунок 6. Диаграмма рассеивания цены и пробега.

8. Прежде чем приступить к анализу, упростим наш проект, сократив количество уровней в переменной color. Исследуя результат выполнения функции `table()` для нашей новой переменной, можно увидеть, что примерно две трети автомобилей имеют консервативные цвета, а одна треть — нет.

```
usedcars$conservative <- usedcars$color %in% c("Black", "Gray", "Silver", "White")
table(usedcars$conservative)

##
## FALSE  TRUE
##    51    99
```

9. Теперь посмотрим на кросс-таблицу, чтобы увидеть, как количество автомобилей консервативных цветов зависит от модели.

```
library("gmodels")
CrossTable(x = usedcars$model, y = usedcars$conservative)

##
##
##   Cell Contents
## |-----|
```

```

## |                               N |
## | Chi-square contribution      |
## |      N / Row Total          |
## |      N / Col Total          |
## |      N / Table Total        |
## |-----|
##
##
## Total Observations in Table: 150
##
##
## usedcars$model | usedcars$conservative |
## FALSE         TRUE   Row Total |
## -----|-----|-----|
##          SE      27      51      78 |
##          0.009    0.004 |
##          0.346    0.654    0.520 |
##          0.529    0.515 |
##          0.180    0.340 |
## -----|-----|-----|
##          SEL      7      16      23 |
##          0.086    0.044 |
##          0.304    0.696    0.153 |
##          0.137    0.162 |
##          0.047    0.107 |
## -----|-----|-----|
##          SES     17     32     49 |
##          0.007    0.004 |
##          0.347    0.653    0.327 |
##          0.333    0.323 |
##          0.113    0.213 |
## -----|-----|-----|
## Column Total    51     99    150 |
##          0.340    0.660 |
## -----|-----|-----|
##
##

```

Раздел второй: «Отчет о выполнении индивидуального задания»

Разработка предиктивной модели для определения вероятности заболевания диабетом.

Целью является прогнозировать вероятность наличия диабета и распределение по ней наблюдаемых объектов по двум классам: 0 - отсутствие и 1 - наличие соответствующего заболевания.

Это может быть полезно медицинским работникам при выявлении пациентов, которые могут быть подвержены риску развития диабета, и при разработке индивидуальных планов лечения.

Описание используемых инструментов

Для достижения поставленных задач использовался язык программирования R и специальное программное обеспечение R Studio.

R — язык программирования для статистической обработки данных и работы с графикой, а также свободная программная среда вычислений с открытым исходным кодом.

Основные особенности R:

- Интерпретируемость. Язык выполняет команды сразу после их ввода, без предварительной компиляции кода. Это делает работу с R интерактивной и позволяет быстро проверять результаты, внося изменения в реальном времени.
- Простота синтаксиса. Язык напоминает естественный и интуитивно понятен. В нём есть четыре примитивных типа данных: логические, числовые (вещественные и целочисленные), символьные и комплексные.
- Возможность создания графики. R позволяет создавать не только статичные, но и интерактивные графики, а также веб-приложения на их основе.

R Studio — это интегрированная среда разработки (IDE) для языка программирования R. Она предоставляет удобный интерфейс для написания кода, анализа данных, визуализации и создания отчетов. R Studio включает редактор кода, консоль, панель вывода графиков, среду просмотра данных и инструменты для управления проектами и пакетами R. Благодаря своей удобной и мощной среде R Studio стал популярным инструментом среди статистиков, аналитиков данных и исследователей для выполнения анализа и моделирования данных.

Описание используемой модели

Для предсказания использована одно из семейств моделей GLM. Существует несколько семейств моделей GLM в зависимости от состава переменной отклика. Для работы выбрана биномиальное семейство. Выбор основан на том, что Биномиальное семейство используется для бинарных переменных отклика (т.е. двух категорий) и предполагает биномиальное распределение. Иными словами, в работе используется логистическая регрессия, которая будет определять зависимую бинарную переменную наличия заболевания от независимых переменных. Независимыми переменными будут служить все остальные переменные в `train_data` в качестве предикторов. Это может включать различные факторы, такие как возраст, индекс массы тела, уровень глюкозы и т.д.

Описание набора данных

Набор данных взят с сайта kaggle. Ссылка для скачивания:

<https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset/data>

Набор данных для прогнозирования диабета представляет собой набор медицинских и демографических данных о пациентах, а также их диабетическом статусе (положительном или отрицательном). Эти данные включают такие характеристики, как возраст, пол, индекс массы тела (ИМТ), наличие гипертонии, наличие заболевания сердца, история курения, уровень HbA1c и глюкозы в крови. Этот набор данных используется для построения модели машинного обучения и прогнозирования диабета у пациентов на основе их истории болезни и демографической информации.

Описание используемых метрик

Чувствительность и специфичность — это показатели, которые используются для оценки работы классификатора.

- **Чувствительность (Recall)** измеряет способность классификатора обнаруживать состояние, когда оно присутствует. (Чувствительность =

Истинные положительные результаты / Истинные положительные результаты + Ложные отрицательные результаты).

- **Специфичность (Precision)** оценивает способность классификатора правильно исключать состояние, когда оно отсутствует. (Специфичность = Истинные отрицательные результаты / Истинные отрицательные результаты + Ложные положительные результаты).
- **Accuracy (точность)** показывает долю правильных предсказаний модели среди всех предсказаний. Значение Accuracy находится в диапазоне от 0 до 1 (или от 0% до 100%). Чем ближе значение к 1 (100%), тем точнее модель предсказывает результаты.
- **F1-score** — гармоническое среднее между Precision (точностью) и Recall (полнотой), которое позволяет сбалансировать их значения. Значение F1-score находится в диапазоне от 0 до 1, где 0 означает наихудший возможный результат, а 1 — идеальную модель с безупречной точностью и полнотой.

Реализация решения

Подключение необходимых пакетов

```
# Libraries
library("caret")
library("ggplot2")
library("ROCR")
set.seed(42)
```

Чтение и предобработка данных

1. Считывание данных из файла.

Загрузим данные из файла CSV с именем “diabetes_prediction_dataset.csv” и сохраним их в переменной my_df. Также посмотрим структуру данных.

```
my_df <- read.csv("diabetes_prediction_dataset.csv")
str(my_df)

## 'data.frame':    100000 obs. of  9 variables:
## $ gender          : chr  "Female" "Female" "Male" "Female" ...
## $ age             : num   80 54 28 36 76 20 44 79 42 32 ...
## $ hypertension    : int    0 0 0 0 1 0 0 0 0 0 ...
```

```
## $ heart_disease      : int  1 0 0 0 1 0 0 0 0 0 ...
## $ smoking_history    : chr  "never" "No Info" "never" "current" ...
## $ bmi                : num  25.2 27.3 27.3 23.4 20.1 ...
## $ HbA1c_level        : num  6.6 6.6 5.7 5 4.8 6.6 6.5 5.7 4.8 5 ...
## $ blood_glucose_level: int  140 80 158 155 155 85 200 85 145 100 ...
## $ diabetes           : int   0 0 0 0 0 0 1 0 0 0 ...
```

2. Преобразование столбцов в факторы.

Некоторые столбцы измерены не в количественных шкалах, их стоит перевести в фактор, что позволит R правильно обрабатывать категориальные переменные.

```
my_df[, c(1, 3, 4, 5, 9)] <- lapply(my_df[, c(1, 3, 4, 5, 9)], as.factor)
str(my_df)

## 'data.frame':    100000 obs. of  9 variables:
## $ gender          : Factor w/ 3 levels "Female","Male",...: 1 1 2 1 2 1 1 1 2 1 .
## ..
## $ age             : num  80 54 28 36 76 20 44 79 42 32 ...
## $ hypertension    : Factor w/ 2 levels "0","1": 1 1 1 1 2 1 1 1 1 1 ...
## $ heart_disease    : Factor w/ 2 levels "0","1": 2 1 1 1 2 1 1 1 1 1 ...
## $ smoking_history  : Factor w/ 6 levels "current","ever",...: 4 5 4 1 1 4 4 5 4 4
## ...
## $ bmi             : num  25.2 27.3 27.3 23.4 20.1 ...
## $ HbA1c_level      : num  6.6 6.6 5.7 5 4.8 6.6 6.5 5.7 4.8 5 ...
## $ blood_glucose_level: int  140 80 158 155 155 85 200 85 145 100 ...
## $ diabetes        : Factor w/ 2 levels "0","1": 1 1 1 1 1 1 2 1 1 1 ...
```

3. Удаление дубликатов и проверка на наличие пропущенных значений.

```
# удалим повторяющиеся записи
my_df <- unique(my_df)
# проверим на пропущенные значения
any(is.na(my_df))

## [1] FALSE
```

Пропущенных значений нет, теперь можно посмотреть краткую сводку о данных и перейти к обработке данных.

```
summary(my_df)

##      gender      age      hypertension heart_disease      smoking_history
## Female:56161   Min.    : 0.00      0:88685      0:92223      current    : 9197
## Male  :39967   1st Qu.:24.00      1: 7461      1: 3923      ever       : 3998
## Other  :   18   Median :43.00                      former     : 9299
##                               Mean  :41.79                      never      :34398
##                               3rd Qu.:59.00                      No Info    :32887
##                               Max.   :80.00                      not current: 6367
##      bmi      HbA1c_level      blood_glucose_level diabetes
```

```
## Min. :10.01 Min. :3.500 Min. : 80.0 0:87664
## 1st Qu.:23.40 1st Qu.:4.800 1st Qu.:100.0 1: 8482
## Median :27.32 Median :5.800 Median :140.0
## Mean :27.32 Mean :5.533 Mean :138.2
## 3rd Qu.:29.86 3rd Qu.:6.200 3rd Qu.:159.0
## Max. :95.69 Max. :9.000 Max. :300.0
```

4. Удаление выбросов методом межквартильного размаха.

Есть подозрения на наличие выбросов в переменной `bmi`. Это можно проверить с помощью графика `boxplot` (См. рисунок 7).

```
boxplot(my_df$bmi, main = "bmi")
```

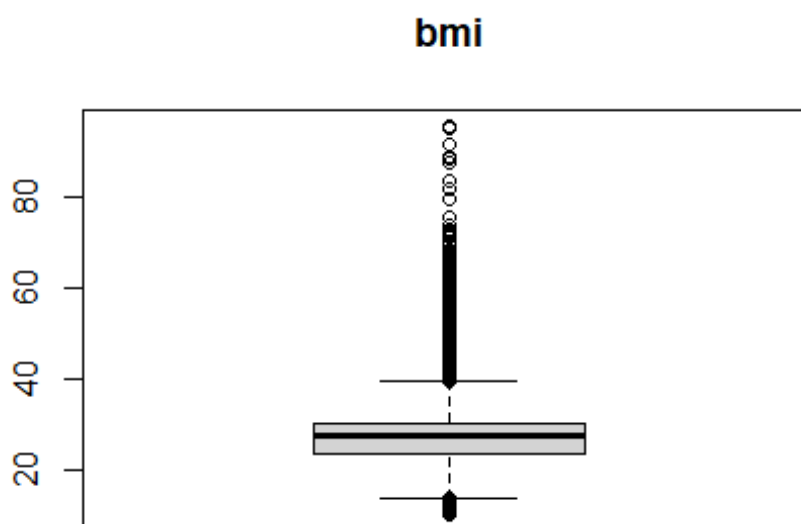


Рисунок 7. Ящик с усами для `bmi`.

Как видно, существует множество значений, выходящих за границу крайних квартилей. Удалим наблюдения с ними для лучшей работы моделей

```
# удаляем выбросы
q_1 <- quantile(my_df$bmi, 0.25)
q_3 <- quantile(my_df$bmi, 0.75)
iqr <- q_3 - q_1
lower_bound <- q_1 - 1.5 * iqr
upper_bound <- q_3 + 1.5 * iqr
my_df <- my_df[my_df$bmi >= lower_bound & my_df$bmi <= upper_bound, ]
```

5. Стандартизация данных.


```
my_df[, -c(1, 3, 4, 5, 9)] <- sapply(my_df[, -c(1, 3, 4, 5, 9)], scale)
head(my_df)
```

##	gender	age	hypertension	heart_disease	smoking_history	bmi
## 1	Female	1.6912937	0	1	never	-0.2332038
## 2	Female	0.5442608	0	0	No Info	0.1737743
## 3	Male	-0.6027722	0	0	never	0.1737743
## 4	Female	-0.2498389	0	0	current	-0.5656648
## 5	Male	1.5148271	1	1	current	-1.1981050
## 6	Female	-0.9557054	0	0	never	0.1737743

##	HbA1c_level	blood_glucose_level	diabetes
## 1	1.0133688	0.05707638	0
## 2	1.0133688	-1.43018178	0
## 3	0.1687918	0.50325382	0
## 4	-0.4881014	0.42889092	0
## 5	-0.6757852	0.42889092	0
## 6	1.0133688	-1.30624360	0

6. Разделение данных на обучающую и тестовую выборки.

В этом шаге данные разделяются на обучающую (70%) и тестовую (30%) выборки.

Функция `createDataPartition` используется для случайного выбора индексов для обучающей выборки, а оставшиеся данные используются для тестовой выборки.

```
# split data
train_index <- createDataPartition(my_df$diabetes, p = 0.7, list = FALSE)
train_data <- my_df[train_index, ]
test_data <- my_df[-train_index, ]
```

Обучение и предсказание модели

```
# train model on split data
model <- glm(diabetes ~ ., data = train_data, family = "binomial")

predictions <- predict(model, newdata = test_data[, c(-9)], type = "response")
```

Выполнив код, мы получили вероятности того, относится ли наблюдаемый объект к классу 1 (имеется диабет). Установив пороговое значение, мы сможем определять сам класс.

Определение порогового значения

Теперь остро встает вопрос определения порогового значения для отнесения объекта к классу. Для этого в своих воспользуемся визуализацией метрик модели.

Для начала отобразим ROC Curve, чтобы убедиться в хорошей точности модели и отсутствии перекошенности (См. рисунок 8).

```
#####  
pred_fit <- prediction(as.numeric(predictions), as.numeric(test_data$diabetes))  
perf_fit <- performance(pred_fit, "tpr", "fpr")  
plot(perf_fit)  
# Add a diagonal reference line  
abline(a = 0, b = 1, col = "red", lty = 2)
```

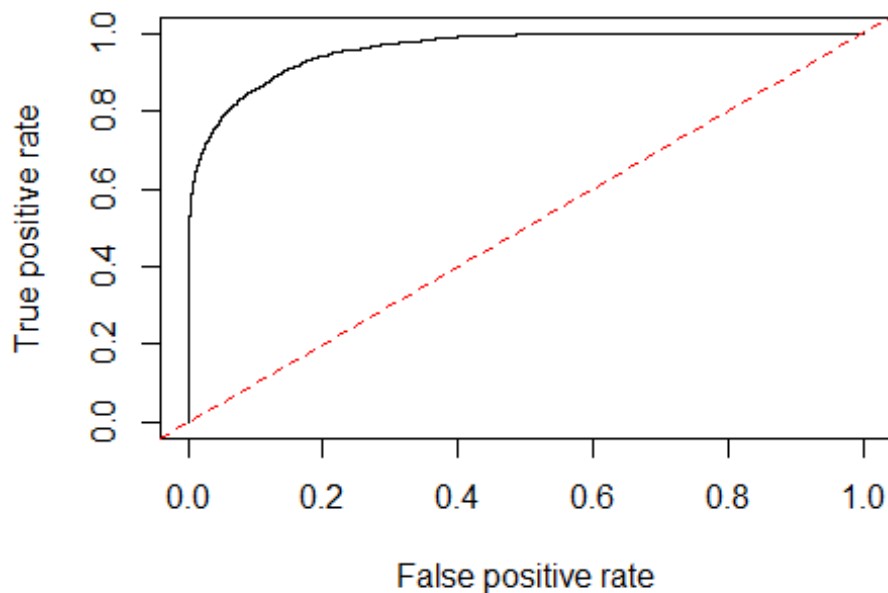


Рисунок 8. ROC кривая.

Теперь отобразим две метрики Specificity и Sensitivity. Их пересечение позволит определить оптимальную границу (См. рисунок 9).

```
#####  
perf3 <- performance(pred_fit, x.measure = "cutoff", measure = "spec")  
perf4 <- performance(pred_fit, x.measure = "cutoff", measure = "sens")  
  
plot(perf3, col = "red", lwd = 2)  
plot(add = T, perf4, col = "green", lwd = 2)  
# Отобразим линию пересечения на координате 0.07  
abline(v = 0.07, lwd = 2)
```

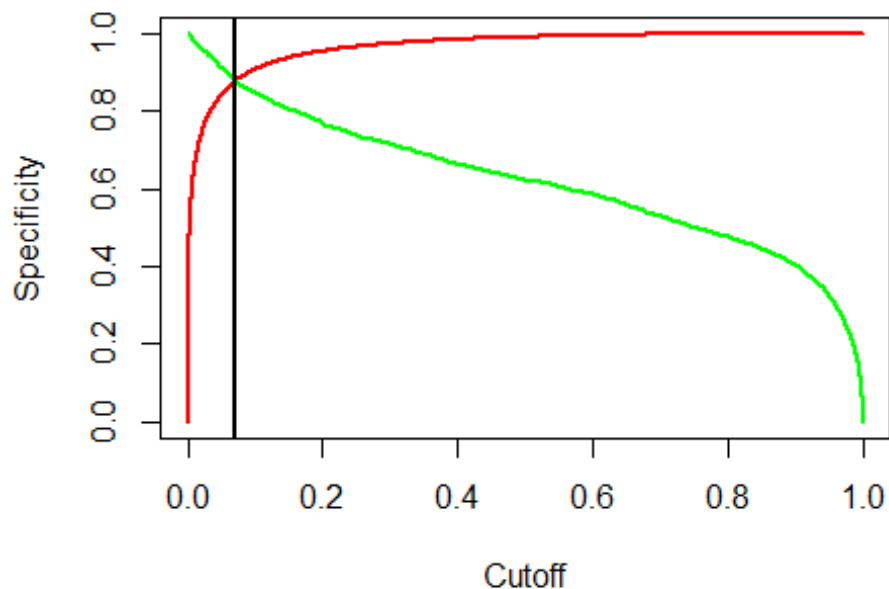


Рисунок 9. Specificity и Sensitivity для разных границ.

По графику мы можем определить пороговое значение, которое, как видно, чуть меньше 0.1. Возьмем приблизительно 0.07. Теперь можем поменять предсказанные вероятности на определенный класс.

```
threshold <- 0.07
```

Определение классов и векторов с метками

```
predicted_classes <- ifelse(predictions > threshold, 1, 0)
```

```
correct <- ifelse(predicted_classes == test_data$diabetes, 1, 0)
```

Использование различных метрик точности модели

```
predicted_classes <- factor(predicted_classes)
test_data$diabetes <- factor(test_data$diabetes)
```

```
confusion_matrix <- confusionMatrix(predicted_classes, test_data$diabetes)
print(confusion_matrix)
```

```
## Confusion Matrix and Statistics
##
```

```
##           Reference
## Prediction      0      1
##           0 21980   252
##           1  3093  1912
##
##           Accuracy : 0.8772
##           95% CI : (0.8732, 0.8811)
##           No Information Rate : 0.9205
##           P-Value [Acc > NIR] : 1
##
##           Kappa : 0.4752
##
##           McNemar's Test P-Value : <2e-16
##
##           Sensitivity : 0.8766
##           Specificity : 0.8835
##           Prevalence : 0.9205
##           Detection Rate : 0.8070
##           Detection Prevalence : 0.8162
##           Balanced Accuracy : 0.8801
##
##           'Positive' Class : 0
##
#####
accuracy <- mean(correct)
f1_score <- confusion_matrix$byClass["F1"]
```

Итак, мы получили Confusion matrix (матрицу ошибок) с различными метриками и две основные: accuracy = 0.8771891 и F1-score = 0.9292887.

Сохранение модели

Сохраним модель вместе с пороговым значением в model.RData.

```
save(model, threshold, file = "model.RData")
```

Заключение

Таким образом, нам удалось получить предиктивную модель, способную определить наличие или отсутствие диабета у наблюдаемого объекта. Для прогнозирования необходимы такие характеристики о пациенте как возраст, пол, индекс массы тела (ИМТ), наличие гипертонии, наличие заболевания сердца, история курения, уровень HbA1c и глюкозы в крови. Наличие этих данных обеспечит высокую вероятность правильной классификации, так как при обучении модели зависимая переменная риска заболевания определялась по каждой из них. Важным замечанием является то, что в расчет не бралась взаимосвязь нескольких независимых переменных, так как она не оказывала значительного влияния на работу модели, а также в целях уменьшения размера модели. Для определения качества работы модели использовались несколько метрик: Accuracy = 0.877; F1-score = 0.929; Sensitivity = 0.876; Specificity = 0.883. Подробное описание данных метрик приведено в введении. Также в реализации решения приведена confusion matrix (матрица ошибок) отображающая количество верных (главная диагональ) и неверных классификаций. Все метрики принимают высокие значения (> 0.85). Следовательно, можно сделать вывод, что модель имеет высокую точность классификации.

В ходе прохождения учебной технологической практики были приобретены необходимые навыки и компетенции, позволившие успешно выполнить поставленные задачи в рамках практической работы.

Список литературы

1. Bioinformatics Institute: Анализ данных в R — stepic.org — URL: <https://stepik.org/course/129/syllabus>
2. Programming language R — URL: <https://www.r-project.org>
3. Wikipedia: R language — URL: [https://ru.wikipedia.org/wiki/R_\(язык_программирования\)](https://ru.wikipedia.org/wiki/R_(язык_программирования))
4. Воронина, В. В. Теория и практика машинного обучения. — URL: <https://e.lanbook.com/book/165053>
5. Selva Prabhakaran: Logistic Regression with R — URL: <https://r-statistics.co/Logistic-Regression-With-R.html>
6. Joao Pedro Lopes: Prediction using Logistic Regression — URL: <https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset/code>